E3 — FUNDAMENTAÇÃO TEÓRICA E TRABALHOS RELACIONADOS

Eduardo Krause (RA 12520157) — Análise e Desenvolvimento de Sistemas — UNIMATER
Orientador: Prof. Me. João Mazzochin — Entrega de 13/10/2026

Este documento traz o rascunho da Seção 2 do artigo, já no formato do template institucional, e o quadro comparativo que posiciona o trabalho frente à literatura. O texto parte das 40 referências e do fichamento entregues na E2. As referências citadas foram conferidas novamente nas páginas dos editores, o que gerou duas correções em relação à E2, registradas nas observações ao final.

2 Fundamentação Teórica e Trabalhos Relacionados

Esta seção apresenta os conceitos necessários à compreensão da metodologia, as tecnologias empregadas na solução e os trabalhos que tratam de problemas próximos ao investigado, encerrando com a comparação que delimita a contribuição pretendida.

2.1 Conceitos Fundamentais

O desenvolvimento guiado por comportamento (*behavior-driven development*, BDD) é uma abordagem de especificação em que o comportamento esperado do sistema é descrito por exemplos concretos, redigidos em linguagem próxima à do negócio e compartilhados entre analistas, desenvolvedores e testadores (SOLIS; WANG, 2011). Esses exemplos são comumente escritos na notação Gherkin, que organiza cada cenário em passos de contexto (Dado), ação (Quando) e resultado esperado (Então), agrupados por funcionalidade (WYNNE; HELLESOY, 2012; CUCUMBER, 2026). No processo de teste descrito pela ISO/IEC/IEEE 29119-2 (2021), casos de teste são derivados de uma base de teste; neste trabalho, a base é composta pelo requisito da tarefa e pela documentação funcional da rotina. Revisões sistemáticas apontam o esforço de elaborar e manter os cenários como a principal dificuldade do BDD (FAROOQ et al., 2023; ARREDONDO-REYES et al., 2023).

Um caso de teste só tem valor se houver um oráculo, mecanismo que decide se o comportamento observado está correto. Barr et al. (2015) denominam problema do oráculo a dificuldade de obter esse mecanismo e distinguem oráculos especificados,

derivados e implícitos, além do recurso ao julgamento humano quando nenhum deles está disponível. Em relatórios de sistemas de gestão, o oráculo é o valor que o relatório deve exibir para a massa de dados existente, hoje apurado manualmente pelo analista. A proposta substitui essa apuração por um oráculo derivado: o valor é obtido pela execução de consultas sobre os dados armazenados.

Convém distinguir dois atributos do valor obtido dessa forma. O valor é **apurado** quando provém da execução de uma consulta sobre os dados armazenados, sendo rastreável e reprodutível; é **validado** quando a consulta representa corretamente a regra de negócio que o relatório implementa. A execução assegura o primeiro atributo, mas não o segundo: uma consulta pode executar sem erro, retornar dados reais e ainda calcular outra coisa. A plataforma garante a apuração; a validação depende do consenso entre modelos, como indicador de confiança, e da conferência do analista, como decisão final.

Modelos de linguagem de grande porte passaram a ser aplicados a praticamente todas as atividades da engenharia de software (HOU et al., 2024), inclusive ao teste (WANG et al., 2024). A limitação central para o problema aqui tratado é a alucinação, isto é, a produção de conteúdo fluente e plausível, porém infiel aos fatos ou à fonte (HUANG et al., 2025). Em engenharia de software, ela se manifesta em artefatos que aparentam correção sem tê-la, agravada pela não determinação das saídas e pela substituição frequente das versões dos modelos, que comprometem a reprodutibilidade (SALLOU; DURIEUX; PANICHELLA, 2024). Uma das estratégias de mitigação é ancorar a geração em conteúdo recuperado de fonte externa no momento da consulta (LEWIS et al., 2020; HUANG et al., 2025), papel que, na solução, cabe ao requisito, à documentação da rotina e ao catálogo do banco de dados.

A apuração dos valores esperados apoia-se na tradução de perguntas em linguagem natural para consultas SQL (*text-to-SQL*). No conjunto Spider (YU et al., 2018), abordagens baseadas em modelos de linguagem superam 85% de acurácia por execução (POURREZA; RAFIEI, 2023; GAO et al., 2024). O desempenho cai quando esquema e dados se aproximam dos corporativos: no BIRD, com 95 bancos e 33,4 GB de dados, o ChatGPT atingiu 40,08% de acurácia por execução, contra 92,96% de

especialistas humanos (LI et al., 2023); no Spider 2.0, com 632 problemas derivados de fluxos de trabalho empresariais, o melhor agente avaliado resolveu 21,3% das tarefas (LEI et al., 2025). A acurácia por execução considera correta a consulta cujo conjunto de resultado coincide com o da consulta de referência. Zhong, Yu e Klein (2020) mostram que esse critério evita os falsos negativos da comparação textual, de 2,5% em média e 8,1% no pior caso, mas que, executado sobre uma única base, admite falsos positivos, quando consultas distintas coincidem por acaso nos dados disponíveis. Neste trabalho, a comparação de conjuntos de resultado é o critério para decidir se dois modelos concordam, e o risco de falso positivo corresponde ao falso acordo: modelos que concordam em uma consulta que não representa a regra de negócio. A taxa de falso acordo mede, portanto, a distância entre valor apurado e valor validado.

Por fim, a decisão por concordância apoia-se em resultados que mostram ganho ao substituir uma única saída pela mais frequente entre várias. Wang et al. (2023) amostram diversos raciocínios de um mesmo modelo e adotam a resposta majoritária, estratégia denominada autoconsistência, com ganho de 17,9 pontos percentuais no conjunto GSM8K. Du et al. (2024) fazem múltiplas instâncias debaterem suas respostas até convergirem, o que reduz afirmações falaciosas; seus experimentos principais usam instâncias de um mesmo modelo, e apenas um experimento reduzido, com 20 problemas, combina modelos de famílias distintas. Como amostras de um mesmo modelo tendem a compartilhar os mesmos erros, a hipótese de que modelos de origens distintas reduzem essa correlação permanece pouco explorada, e é examinada neste trabalho por meio da taxa de falso acordo.

2.2 Tecnologias Envolvidas

Três modelos de linguagem de famílias distintas são empregados: Claude, da Anthropic; Gemini, do Google; e um modelo de pesos abertos da família Llama, acessado pela infraestrutura de inferência da Groq. A distinção de origem é condição para a validade do consenso, e o número três decorre da necessidade de desempate. As versões exatas de cada modelo serão fixadas e registradas antes dos experimentos, em atenção ao risco de reprodutibilidade apontado por Sallou, Durieux e Panichella (2024).

O sistema gerenciador de banco de dados é o MySQL, no qual reside a base de homologação do sistema estudado e do qual provêm os valores de referência. O esquema é extraído das visões do INFORMATION_SCHEMA, que expõem tabelas, colunas, tipos, chaves e restrições de integridade referencial (ORACLE, 2026), e o plano de execução é obtido pelo comando EXPLAIN antes de cada execução. Os cenários seguem a gramática Gherkin (CUCUMBER, 2026), verificada por analisador sintático antes da entrega ao analista.

A camada de orquestração é implementada em Java com o *framework* Spring Boot, escolha que oferece acesso nativo a bancos relacionais e execução concorrente das chamadas aos três modelos; a interface é desenvolvida em React. Requisitos e documentação são lidos das interfaces de programação do Jira e do Confluence, ferramentas de gestão de tarefas e de documentação adotadas pela equipe. Ambas as camadas são empacotadas em um único artefato executado localmente, pois as credenciais de acesso aos modelos e ao banco não podem residir no navegador e o banco corporativo não deve ser exposto à rede externa.

2.3 Trabalhos Relacionados

O Quadro 1 compara os trabalhos mais próximos segundo os critérios que distinguem a proposta: as fontes de entrada, o artefato produzido, a apuração de valores esperados sobre dados reais, o uso de mais de um modelo e o contexto em que a solução foi avaliada.

Quadro 1 – Comparação entre os trabalhos relacionados e a proposta

Trabalho	**Entrada**	**Saída**	**Valor esperado apurado em dados reais**	**Uso de vários modelos**	**Contexto e validação**
Karpurapu et al. (2024)	*User stories* públicas	Cenários Gherkin	Não	Compara quatro modelos, sem combiná-los	Acadêmico; validação sintática dos arquivos gerados
Fernandes et al. (2025)	Descrições livres de casos de teste	Cenários Gherkin	Não	Compara sete modelos, sem combiná-los	Sistema em produção, dez casos; similaridade textual com referência de especialista
Paduraru, Zavelca e Stefanescu (2025)	Descrição em linguagem natural e passos já implementados	Passos Gherkin vinculados ao código	Não	Um modelo ajustado, com agentes	Industrial (jogos); testes sintéticos e estudo qualitativo com profissionais
Ferreira et al. (2025)	*User story* do Jira e HTML das	Gherkin e roteiros Cypress	Não	Um modelo	Industrial (web); utilidade percebida por profissionais

Trabalho	Entrada	Saída	Valor esperado apurado em dados reais	Uso de vários modelos	Contexto e validação
	páginas				
Fonseca, Lima e Faria (2025)	Tarefa do Jira e alterações de código	Gherkin, Page Objects e roteiros de interface	Não	Um modelo por etapa, sem votação	Industrial (aplicativo móvel); sintaxe, execução e percepção de profissionais
Garg e Sharma (2023)	Solicitação do usuário	Casos e cenários de teste	Não	Um modelo	ERP; sem avaliação empírica
Mendoza et al. (2024)	Cenários BDD	Dados de teste sintéticos	Não; gera massa nova	Compara quatro ferramentas, sem combiná-las	Cinco cenários ilustrativos; avaliação qualitativa
Este trabalho	**Requisito do Jira, documentação da rotina e esquema do banco**	**Cenários Gherkin com valores esperados e consultas anexadas**	**Sim, por execução em homologação**	**Três famílias, com decisão por consenso**	**ERP em uso (estoque e vendas); avaliação por analistas e contra conjunto de referência**

Fonte: elaborado pelo autor (2026).

Os dois primeiros trabalhos medem a capacidade de modelos de converter textos em Gherkin. Karpurapu et al. (2024) comparam quatro modelos sobre *user stories* públicas e verificam apenas a sintaxe dos arquivos gerados; Fernandes et al. (2025) comparam sete modelos e três estratégias de *prompt* sobre casos de um sistema em produção, medindo a similaridade com cenários escritos por especialista. Ambos confirmam a viabilidade da geração, mas tratam os modelos como alternativas a escolher, e não como fontes a combinar, e nenhum verifica se o resultado esperado descrito no cenário está correto.

Os três trabalhos seguintes levam a geração ao ambiente industrial. Paduraru, Zavelca e Stefanescu (2025) vinculam passos Gherkin a implementações existentes, com humano no ciclo; Ferreira et al. (2025) partem de *user stories* do Jira e chegam a roteiros Cypress; e Fonseca, Lima e Faria (2025), o trabalho mais próximo, geram cenários, Page Objects e roteiros executáveis a partir de tarefas do Jira e das alterações de código, com 93,3% dos cenários sintaticamente corretos. A integração com a ferramenta de gestão e a geração a partir do requisito, portanto, já estão cobertas pela literatura, e não constituem diferencial desta proposta. O que permanece em aberto é o uso da documentação funcional como segunda fonte, capaz de cobrir os comportamentos que a rotina já deveria apresentar e que o requisito não menciona, e

qualquer forma de oráculo de dados: nesses trabalhos, a verificação incide sobre a sintaxe e a execução do roteiro, nunca sobre o resultado de negócio.

Os dois últimos tocam o domínio e os dados. Garg e Sharma (2023) aplicam IA generativa à modelagem de testes de ERP, mas sem avaliação empírica. Mendoza et al. (2024) comparam ferramentas na geração de dados de teste a partir de cenários BDD, o caminho inverso ao proposto: geram massa sintética, enquanto este trabalho parte da massa existente e apura o valor que o sistema deve apresentar sobre ela.

Diferentemente dos trabalhos apresentados, este trabalho confronta a saída dos modelos com os dados efetivamente armazenados: o valor esperado de cada cenário é apurado pela execução, em ambiente somente leitura, de consultas geradas de forma independente por modelos de três famílias, e só é proposto como referência quando ao menos dois resultados coincidem, cabendo ao analista validá-lo. Articula, assim, duas linhas que a literatura trata em separado, a geração de cenários e a tradução para SQL, e avalia empiricamente essa articulação em um sistema de gestão em uso, medindo inclusive quando o consenso engana.

Referências

ARREDONDO-REYES, V. M.; DOMÍNGUEZ-ISIDRO, S.; SÁNCHEZ-GARCÍA, Á. J.; OCHARÁN-HERNÁNDEZ, J. O. Benefits and Challenges of the Behavior-Driven Development: A Systematic Literature Review. In: INTERNATIONAL CONFERENCE IN SOFTWARE ENGINEERING RESEARCH AND INNOVATION (CONISOFT), 11., 2023. **Anais** [...]. IEEE, 2023. p. 45-54. DOI: 10.1109/CONISOFT58849.2023.00016.

BARR, E. T.; HARMAN, M.; McMINN, P.; SHAHBAZ, M.; YOO, S. The Oracle Problem in Software Testing: A Survey. **IEEE Transactions on Software Engineering**, v. 41, n. 5, p. 507-525, 2015. DOI: 10.1109/TSE.2014.2372785.

CUCUMBER. **Gherkin Reference**. 2026. Disponível em: https://cucumber.io/docs/gherkin/reference/. Acesso em: 9 set. 2026.

DU, Y.; LI, S.; TORRALBA, A.; TENENBAUM, J. B.; MORDATCH, I. Improving Factuality and Reasoning in Language Models through Multiagent Debate. In: INTERNATIONAL CONFERENCE ON MACHINE LEARNING (ICML), 41., 2024, Viena. **Proceedings** [...]. PMLR, 2024. v. 235, p. 11733-11763.

FAROOQ, M. S.; OMER, U.; RAMZAN, A.; RASHEED, M. A.; ATAL, Z. Behavior Driven Development: A Systematic Literature Review. **IEEE Access**, v. 11, p. 88008-88024, 2023. DOI: 10.1109/ACCESS.2023.3302356.

FERNANDES, H.; PERKUSICH, M.; ALBUQUERQUE, D.; SILVA, I.; SANTOS, D. F. S.; GORGÔNIO, K.; PERKUSICH, A. A Comparative Study of LLMs for Gherkin Generation. In: SIMPÓSIO BRASILEIRO DE ENGENHARIA DE SOFTWARE (SBES), 39., 2025, Recife. **Anais** [...]. Porto Alegre: Sociedade Brasileira de Computação, 2025. p. 171-181. DOI: 10.5753/sbes.2025.9888.

FERREIRA, M.; VIEGAS, L.; FARIA, J. P.; LIMA, B. Acceptance Test Generation with Large Language Models: An Industrial Case Study. In: IEEE/ACM INTERNATIONAL CONFERENCE ON AUTOMATION OF SOFTWARE TEST (AST), 2025. **Proceedings** [...]. IEEE, 2025. DOI: 10.1109/AST66626.2025.00007.

FONSECA, P. L.; LIMA, B.; FARIA, J. P. Streamlining Acceptance Test Generation for Mobile Applications Through Large Language Models: An Industrial Case Study. In: IEEE/ACM INTERNATIONAL CONFERENCE ON AUTOMATED SOFTWARE ENGINEERING (ASE), 40., 2025. **Proceedings** [...]. IEEE, 2025. DOI: 10.1109/ASE63991.2025.00273.

GAO, D.; WANG, H.; LI, Y.; SUN, X.; QIAN, Y.; DING, B.; ZHOU, J. Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation. **Proceedings of the VLDB Endowment**, v. 17, n. 5, p. 1132-1145, 2024. DOI: 10.14778/3641204.3641221.

GARG, A.; SHARMA, D. Generative AI for Software Test Modelling with a focus on ERP Software. In: INTERNATIONAL CONFERENCE ON ADVANCES IN COMPUTATION, COMMUNICATION AND INFORMATION TECHNOLOGY (ICAICCIT), 2023, Faridabad. **Proceedings** [...]. IEEE, 2023. p. 187-193. DOI: 10.1109/ICAICCIT60255.2023.10466102.

HOU, X.; ZHAO, Y.; LIU, Y.; YANG, Z.; WANG, K.; LI, L.; LUO, X.; LO, D.; GRUNDY, J.; WANG, H. Large Language Models for Software Engineering: A Systematic Literature Review. **ACM Transactions on Software Engineering and Methodology**, v. 33, n. 8, art. 220, 2024. DOI: 10.1145/3695988.

HUANG, L.; YU, W.; MA, W.; ZHONG, W.; FENG, Z.; WANG, H.; CHEN, Q.; PENG, W.; FENG, X.; QIN, B.; LIU, T. A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions. **ACM Transactions on Information Systems**, v. 43, n. 2, art. 42, p. 1-55, 2025. DOI: 10.1145/3703155.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/IEC/IEEE 29119-2:2021**: Software and systems engineering — Software testing — Part 2: Test processes. 2. ed. Genebra: ISO, 2021.

KARPURAPU, S.; MYNENI, S.; NETTUR, U.; GAJJA, L. S.; BURKE, D.; STIEHM, T.; PAYNE, J. Comprehensive Evaluation and Insights Into the Use of Large Language Models in the Automation of Behavior-Driven Development Acceptance Test Formulation. **IEEE Access**, v. 12, p. 58715-58730, 2024. DOI: 10.1109/ACCESS.2024.3391815.

LEI, F.; CHEN, J.; YE, Y.; CAO, R.; SHIN, D.; SU, H.; SUO, Z.; GAO, H.; HU, W.; YIN, P.; ZHONG, V.; XIONG, C.; SUN, R.; LIU, Q.; WANG, S.; YU, T. Spider 2.0: Evaluating

Language Models on Real-World Enterprise Text-to-SQL Workflows. In: INTERNATIONAL CONFERENCE ON LEARNING REPRESENTATIONS (ICLR), 13., 2025, Singapura. **Proceedings** [...]. OpenReview.net, 2025.

LEWIS, P.; PEREZ, E.; PIKTUS, A.; PETRONI, F.; KARPUKHIN, V.; GOYAL, N.; KÜTTLER, H.; LEWIS, M.; YIH, W.; ROCKTÄSCHEL, T.; RIEDEL, S.; KIELA, D. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In: ADVANCES IN NEURAL INFORMATION PROCESSING SYSTEMS (NeurIPS), 33., 2020. **Proceedings** [...]. Curran Associates, 2020. p. 9459-9474.

LI, J.; HUI, B.; QU, G.; YANG, J.; LI, B.; LI, B.; WANG, B.; QIN, B.; GENG, R.; HUO, N.; ZHOU, X.; MA, C.; LI, G.; CHANG, K.; HUANG, F.; CHENG, R.; LI, Y. Can LLM Already Serve as a Database Interface? A BIg Bench for Large-Scale Database Grounded Text-to-SQLs. In: ADVANCES IN NEURAL INFORMATION PROCESSING SYSTEMS (NeurIPS), 36., 2023. **Proceedings** [...]. Datasets and Benchmarks Track. Curran Associates, 2023.

MENDOZA, I.; SILVA FILHO, F.; MEDEIROS, G.; PAES, A.; NEVES, V. O. Comparative Analysis of Large Language Model Tools for Automated Test Data Generation from BDD. In: SIMPÓSIO BRASILEIRO DE ENGENHARIA DE SOFTWARE (SBES), 38., 2024, Curitiba. **Anais** [...]. Porto Alegre: Sociedade Brasileira de Computação, 2024. p. 280-290. DOI: 10.5753/sbes.2024.3423.

ORACLE. **MySQL 8.4 Reference Manual**: INFORMATION_SCHEMA Tables. 2026. Disponível em: https://dev.mysql.com/doc/refman/8.4/en/information-schema.html. Acesso em: 9 set. 2026.

PADURARU, C.; ZAVELCA, M.; STEFANESCU, A. Agentic AI for Behavior-Driven Development Testing Using Large Language Models. In: INTERNATIONAL CONFERENCE ON AGENTS AND ARTIFICIAL INTELLIGENCE (ICAART), 17., 2025. **Proceedings** [...]. SciTePress, 2025. v. 2, p. 805-815. DOI: 10.5220/0013374400003890.

POURREZA, M.; RAFIEI, D. DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction. In: ADVANCES IN NEURAL INFORMATION PROCESSING SYSTEMS (NeurIPS), 36., 2023. **Proceedings** [...]. Curran Associates, 2023. p. 36339-36348.

SALLOU, J.; DURIEUX, T.; PANICHELLA, A. Breaking the Silence: the Threats of Using LLMs in Software Engineering. In: INTERNATIONAL CONFERENCE ON SOFTWARE ENGINEERING: NEW IDEAS AND EMERGING RESULTS (ICSE-NIER), 46., 2024, Lisboa. **Proceedings** [...]. New York: ACM, 2024. p. 102-106. DOI: 10.1145/3639476.3639764.

SOLIS, C.; WANG, X. A Study of the Characteristics of Behaviour Driven Development. In: EUROMICRO CONFERENCE ON SOFTWARE ENGINEERING AND ADVANCED APPLICATIONS (SEAA), 37., 2011. **Anais** [...]. IEEE, 2011. p. 383-387. DOI: 10.1109/SEAA.2011.76.

WANG, J.; HUANG, Y.; CHEN, C.; LIU, Z.; WANG, S.; WANG, Q. Software Testing With Large Language Models: Survey, Landscape, and Vision. **IEEE Transactions on Software Engineering**, v. 50, n. 4, p. 911-936, 2024. DOI: 10.1109/TSE.2024.3368208.

WANG, X.; WEI, J.; SCHUURMANS, D.; LE, Q. V.; CHI, E. H.; NARANG, S.; CHOWDHERY, A.; ZHOU, D. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In: INTERNATIONAL CONFERENCE ON LEARNING REPRESENTATIONS (ICLR), 11., 2023, Kigali. **Proceedings** [...]. OpenReview.net, 2023.

WYNNE, M.; HELLESOY, A. **The Cucumber Book**: Behaviour-Driven Development for Testers and Developers. Raleigh: Pragmatic Bookshelf, 2012.

YU, T.; ZHANG, R.; YANG, K.; YASUNAGA, M.; WANG, D.; LI, Z.; MA, J.; LI, I.; YAO, Q.; ROMAN, S.; ZHANG, Z.; RADEV, D. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. In: CONFERENCE ON EMPIRICAL METHODS IN NATURAL LANGUAGE PROCESSING (EMNLP), 2018, Bruxelas. **Proceedings** [...]. Association for Computational Linguistics, 2018. p. 3911-3921. DOI: 10.18653/v1/D18-1425.

ZHONG, R.; YU, T.; KLEIN, D. Semantic Evaluation for Text-to-SQL with Distilled Test Suites. In: CONFERENCE ON EMPIRICAL METHODS IN NATURAL LANGUAGE PROCESSING (EMNLP), 2020. **Proceedings** [...]. Association for Computational Linguistics, 2020. p. 396-411. DOI: 10.18653/v1/2020.emnlp-main.29.

Observações sobre esta entrega

1. **Correção em relação à E2.** O fichamento afirmava que Du et al. (2024) usam apenas instâncias de um mesmo modelo. Na releitura, o artigo traz também um experimento reduzido, com 20 problemas do GSM8K, que combina ChatGPT e Bard. O texto acima foi redigido de acordo; a lacuna continua existindo, mas foi enunciada com mais cuidado.
2. **Veículos confirmados.** Fonseca, Lima e Faria (2025) foi publicado no ASE 2025 e Ferreira et al. (2025) no AST 2025; as referências deixaram de apontar para os pré-prints. O DOI de Mendoza et al. (2024) é 10.5753/sbes.2024.3423. O arquivo .bib desta entrega já traz as três correções.
3. **Extensão.** O texto da Seção 2 soma cerca de 1.600 palavras, o equivalente a pouco mais de três páginas no formato institucional, sem contar o quadro. Na consolidação do artigo, a subseção 2.1 é a candidata natural a enxugamento.

4. **Ajuste da proposta.** Os sete pontos da avaliação de 15/09 foram tratados nesta entrega, na E4 e na E5. O documento TCC1_AjustesProposta, enviado junto, lista cada ajuste, quem o pediu e onde foi atendido, e que entra como apêndice da versão final do TCC I. Nesta seção, o ajuste visível é a distinção entre valor apurado e valor validado (2.1), pedida por Rafael Rodrigues Barbosa.